Retrieving Data

Agenda

Delete statement

Select statement

DELETE Statement

The DELETE statement is used to delete existing records in a table.

- Syntax
  DELETE FROM table_name WHERE condition;
- Example
  DELETE FROM Persons;
  DELETE FROM Persons WHERE City='Frankfurt';

Select Statement

- The SELECT statement is used to select data from a database.
- Syntax
  - SELECT * FROM table_name;
  - SELECT column1, column2, ... FROM table_name;
- Example

```
SELECT * FROM Customers;
SELECT CustomerName, City, Country FROM Customers;
```

Select Statement

1. Create departments Table

```
CREATE TABLE departments (
id SERIAL PRIMARY KEY,
department_name VARCHAR(50)
);
```

```
INSERT INTO departments (department_name) VALUES
('IT'),
('HR'),
('Finance'),
('Marketing');
```

Select Statement

2. Create employees Table

```
CREATE TABLE employees (
id SERIAL PRIMARY KEY,
name VARCHAR(50),
age INT,
salary NUMERIC,
dep_id INT,
FOREIGN KEY (dep_id) REFERENCES departments(id)
);
```

```
INSERT INTO employees (name, age, salary, dep_id)
VALUES
('Ahmed', 30, 8000, 1),
('Sara', 28, 6000, 1),
('Mohamed', 35, 7000, 2),
('Mona', 32, 9000, 3),
('Ali', 26, 5000, 2),
('Nour', 29, 7500, 4),
('Hassan', 40, 10000, 3);
```

Select Statement

1. Simple SELECT

```
SELECT name, salary
FROM employees;
```

2. Select Specific Columns

```
SELECT name, salary
FROM employees;
```

Select Statement

2. SELECT with WHERE

WHERE is a clause in a SELECT query that **specifies conditions that rows must meet to be included in the result.**

```
SELECT * FROM employees WHERE department = 'IT';
```

Select Statement

3. CASE WHEN

CASE WHEN is an expression that **checks conditions and returns a value based on which condition is true.**

```sql
SELECT
    name,
    salary,
    CASE
        WHEN salary >= 10000 THEN 'High'
        WHEN salary >= 5000 THEN 'Medium'
        ELSE 'Low'
    END AS salary_level
FROM employees;
```

Select Statement

4. IN

IN is an operator that allows you to **specify multiple possible values in a condition** instead of using many OR statements.

```
SELECT *
FROM employees
WHERE department IN ('IT', 'HR', 'Finance');
```

Select Statement

5. NOT IN

NOT IN is an operator used in a WHERE clause to **select rows where the column value is not in a specified list of values.**

```
SELECT *
FROM employees
WHERE department NOT IN ('HR', 'Admin');
```

Select Statement

6. BETWEEN

BETWEEN is an operator used in the WHERE clause to **select values within a given range (inclusive).**

```
SELECT *
FROM employees
WHERE salary BETWEEN 5000 AND 10000;
```

Select Statement

7. LIKE

Case-sensitive pattern matching.
LIKE is an operator used in the WHERE clause to **find rows that match a specified pattern.**

```
SELECT name FROM employees WHERE name LIKE 'A%';
```

Wildcards Used with LIKE

```
SELECT column_name FROM table_name WHERE column_name LIKE pattern;
```

Select Statement

7. LIKE

1. Names starting with "A"

```
SELECT * FROM students WHERE name LIKE 'A%';
```

2. Names ending with "n"

```
SELECT * FROM students WHERE name LIKE '%n';
```

Select Statement

7. LIKE

3. Names containing "am"

SQL

```sql
SELECT *
FROM students
WHERE name LIKE '%am%';
```

4. Exactly one character before "a"

SQL

```sql
SELECT *
FROM students
WHERE name LIKE '_a%';
```

Summary

- `LIKE` is used for **pattern matching in text**
- `%` = any number of characters
- `_` = one character

Select Statement

8. ILIKE (PostgreSQL case-insensitive LIKE)

ILIKE is similar to LIKE, but it **ignores uppercase and lowercase letters** when searching for patterns.

```
SELECT *
FROM employees
WHERE name ILIKE 'a%';
```

Matches **A, a**

Select Statement

9. AND

AND allows you to filter rows that satisfy **more than one condition at the same time.**

```
SELECT *
FROM employees
WHERE department = 'IT'
AND salary > 6000;
```

Select Statement

10. OR

OR allows you to filter rows that satisfy **at least one of several conditions.**

```
SELECT *
FROM employees
WHERE department = 'IT'
OR department = 'HR';
```

Select Statement

11. NOT

NOT reverses the meaning of a condition. If the condition is true, NOT makes it false, and vice versa.

```
SELECT *
FROM employees
WHERE NOT department = 'HR';
```

Select Statement

12. IS NULL

IS NULL tests whether a column contains a **NULL value**, which represents **no data** or **unknown data** in SQL.

```
SELECT *
FROM employees
WHERE manager_id IS NULL;
```

Select Statement

13. IS NOT NULL

IS NOT NULL tests whether a column contains a **non-NULL value**. It is the opposite of IS NULL.

```
SELECT *
FROM employees
WHERE manager_id IS NOT NULL;
```

Select Statement

14. EXISTS

EXISTS tests for the **existence of rows returned by a subquery** and is often used in conditional filtering..

```
SELECT column_name
FROM table_name
WHERE EXISTS (subquery);
```

```
SELECT name
FROM employees e
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.employee_id = e.id
);
```

- The subquery checks if there are **any orders for the employee**.
- EXISTS returns **TRUE** if the employee has at least one order.
- The main query then selects only those employees.

Select Statement

15. Not EXISTS

NOT EXISTS tests for the **absence of rows** returned by a subquery. It returns **TRUE** if the subquery returns **no rows**, and **FALSE** if it returns **any row**.

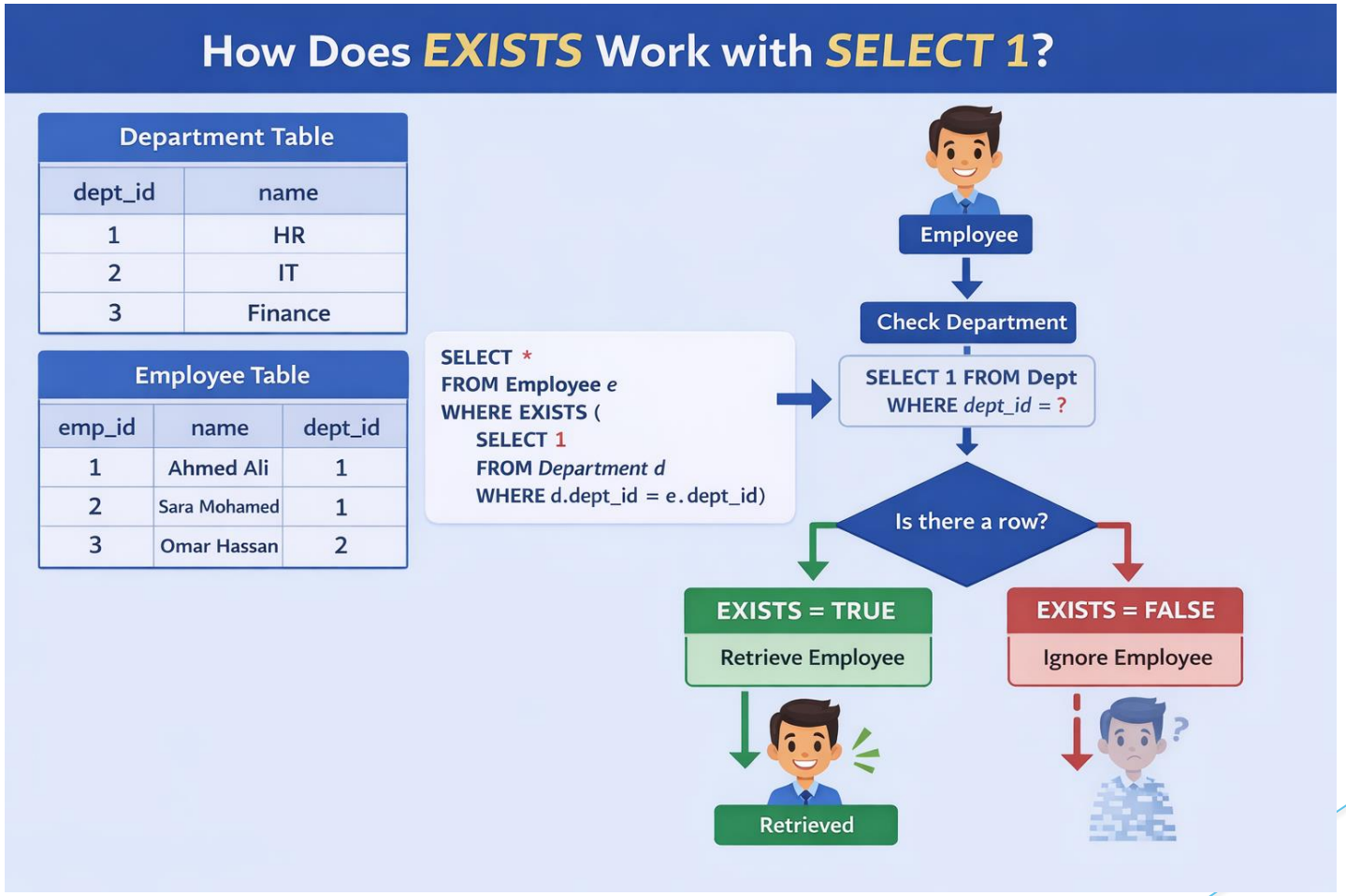
```
SELECT column_name
FROM table_name
WHERE NOT EXISTS (subquery);
```

```
SELECT name
FROM employees e
WHERE NOT EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.employee_id = e.id
);
```

- The subquery checks for orders linked to each employee.
- NOT EXISTS returns **TRUE** for employees who have **no orders**.
- The main query selects only those employees.

Select Statement

EXISTS VS. Not EXISTS



Select Statement

16. Combined Example

```
SELECT name, department, salary
FROM employees
WHERE department IN ('IT','Finance')
AND salary BETWEEN 5000 AND 12000
AND name ILIKE 'a%';
```

Select Statement

In **PostgreSQL**, SIMILAR TO is like a mix between LIKE and regular expressions. It allows **pattern matching using SQL-standard regular expressions**. It's more powerful than LIKE but simpler than full POSIX regex (~).

```
column SIMILAR TO 'pattern'
```

- `%` → matches 0 or more characters
- `_` → matches exactly 1 character
- `|` → OR
- `()` → grouping

```
SELECT name
FROM employees
WHERE name SIMILAR TO '(A|M)%';
```

Explanation:

- `(A|M)` → A or M
- `%` → any characters after

Select Statement

SIMILAR TO

2.2 Names with exactly 5 letters

SQL

```sql
SELECT name
FROM employees
WHERE name SIMILAR TO '_____';
```

Explanation:

- Each `_` is one character → 5 underscores = 5 characters

2.3 Names ending with `a` or `d`

SQL

```sql
SELECT name
FROM employees
WHERE name SIMILAR TO '%(a|d)';
```

- `%` → anything before
- `(a|d)` → ends with `a` or `d`

Select Statement

SIMILAR TO

2.4 Age 28, 29, or 30

SQL

```sql
SELECT name, age
FROM employees
WHERE age::text SIMILAR TO '(28|29|30)';
```

`age::text` is needed because `SIMILAR TO` works on strings.

2.5 Multiple Departments in one pattern

SQL

```sql
SELECT name
FROM employees e
JOIN departments d ON e.dep_id = d.id
WHERE d.department_name SIMILAR TO '(IT|HR)';
```

- Matches employees in IT or HR

- SIMILAR TO is **case-sensitive**. Use ILIKE for case-insensitive matching.
- It's **SQL-standard**, unlike ~ which is PostgreSQL-specific.
- Full regex with ~ or ~* is more flexible.

Select Statement

PostgreSQL Regex Operators
In **PostgreSQL**, regex operators allow powerful pattern matching beyond LIKE or SIMILAR TO. They work with **POSIX-style regular expressions.**
Here's a clear guide with examples using your employees table.

Operator	Description	Example
`~`	Matches regex (case-sensitive)	`'Ahmed' ~ '^A'` → true
`~*`	Matches regex (case-insensitive)	`'ahmed' ~* '^A'` → true
`!~`	Does **not** match regex (case-sensitive)	`'Sara' !~ '^A'` → true
`!~*`	Does **not** match regex (case-insensitive)	`'sara' !~* '^A'` → true

Select Statement

PostgreSQL Regex Operators

2.1 Names starting with `A` (case-sensitive)

SQL

```sql
SELECT name
FROM employees
WHERE name ~ '^A';
```

Result: `Ahmed`

2.2 Names starting with `a` or `A` (case-insensitive)

SQL

```sql
SELECT name
FROM employees
WHERE name ~* '^a';
```

Result: `Ahmed`, `Ali`

2.3 Names ending with `d`

SQL

```sql
SELECT name
FROM employees
WHERE name ~ 'd$';
```

Result: `Ahmed`, `Mohamed`

2.4 Names containing exactly 4 letters

SQL

```sql
SELECT name
FROM employees
WHERE name ~ '^.{4}$';
```

- `.` → any character
- `{4}` → exactly 4 characters
- `^` → start of string
- `$` → end of string

Result: `Mona`

Select Statement

PostgreSQL Regex Operators

2.5 Names containing `a` or `e` anywhere

SQL

```sql
SELECT name
FROM employees
WHERE name ~ '[ae]';
```

- `[ae]` → matches `a` or `e`

2.6 Names NOT starting with `A` (case-insensitive)

SQL

```sql
SELECT name
FROM employees
WHERE name !~* '^A';
```

Result: `Sara`, `Mona`, `Mohamed`, `Nour`, `Hassan`

Select Statement

PostgreSQL Regex Operators

✅ Tips

1. Use `~` for **case-sensitive** and `~*` for **case-insensitive.**
2. Use `!~` and `!~*` to filter rows that **don't match** a pattern.
3. For more advanced patterns, you can use:
   - `^` → start of string
   - `$` → end of string
   - `.` → any single character
   - `*` → zero or more repetitions
   - `+` → one or more repetitions
   - `{n,m}` → n to m repetitions
   - `[]` → character class
   - `|` → OR

Select Statement

DISTINCT

In **PostgreSQL**, the DISTINCT keyword is used to **remove duplicate rows** from the result of a SELECT query. You can use it on one column, multiple columns, or even with aggregate functions.

```
SELECT DISTINCT dep_id
FROM employees;
```

Select Statement

DISTINCT

In **PostgreSQL**, the DISTINCT keyword is used to **remove duplicate rows** from the result of a SELECT query. You can use it on one column, multiple columns, or even with aggregate functions.

```
SELECT DISTINCT dep_id
FROM employees;
```

Select Statement

DISTINCT

2. DISTINCT on Multiple Columns

Get unique combinations of `department` and `salary`:

```sql
SELECT DISTINCT dep_id, salary
FROM employees;
```

Removes duplicate pairs of `dep_id` and `salary`.

Select Statement

Calculated Columns

In **PostgreSQL**, a **calculated column** (also called a derived or computed column) is a column in a SELECT statement that **doesn't exist in the table**, but is **computed on the fly** using expressions, arithmetic, functions, or case logic.

```sql
SELECT
    name,
    salary,
    salary * 1.10 AS salary_with_bonus
FROM employees;
```

Select Statement

PostgreSQL String Functions in SELECT

Here's a **comprehensive guide to PostgreSQL string functions** you can use in SELECT statements, with examples using the employees table (name column) and departments table.

1. `UPPER()` and `LOWER()`

Convert strings to uppercase or lowercase.

SQL

```sql
SELECT name,
       UPPER(name) AS name_upper,
       LOWER(name) AS name_lower
FROM employees;
```

2. `INITCAP()`

Capitalize the first letter of each word.

SQL

```sql
SELECT name, INITCAP(name) AS name_capitalized
FROM employees;
```

Result: `Ahmed`, `Sara` (already capitalized, but works for all lowercase names).

Select Statement

PostgreSQL String Functions in SELECT

3. `LENGTH()`

Get the number of characters in a string.

SQL

```sql
SELECT name, LENGTH(name) AS name_length
FROM employees;
```

4. `SUBSTRING()`

Extract part of a string.

SQL

```sql
SELECT name,
       SUBSTRING(name FROM 1 FOR 3) AS first_3_letters
FROM employees;
```

- `FROM 1 FOR 3` → start at position 1, take 3 characters
- Example: `Ahmed` → `Ahm`

5. `CONCAT()` and `||`

Combine strings.

SQL

```sql
SELECT name,
       CONCAT(name, ' works in ', dep_id) AS info
FROM employees;
```

Or using `||`:

SQL

```sql
SELECT name || ' works in department ' || dep_id AS info
FROM employees;
```

Select Statement

PostgreSQL String Functions in SELECT

6. `TRIM()`, `LTRIM()`, `RTRIM()`

Remove spaces or characters.

SQL

```sql
SELECT '  Ahmed  ' AS original,
       TRIM('  ' FROM '  Ahmed  ') AS trimmed,
       LTRIM('  Ahmed  ') AS left_trimmed,
       RTRIM('  Ahmed  ') AS right_trimmed;
```

7. `REPLACE()`

Replace a substring.

SQL

```sql
SELECT name,
       REPLACE(name, 'a', '@') AS replaced_name
FROM employees;
```

- `Ahmed` → `Ahmed` (no lowercase `a`),
- `Sara` → `S@r@`

8. `POSITION()`

Find position of substring.

SQL

```sql
SELECT name,
       POSITION('a' IN name) AS first_a_position
FROM employees;
```

- Returns index of first occurrence (`1`-based).

Select Statement

PostgreSQL Type Casting

In **PostgreSQL**, **type casting** lets you convert a value from one data type to another. This is useful when comparing different types, formatting output, or performing calculations. PostgreSQL supports **explicit** and **implicit** type casting.

1. Explicit Type Casting

You can use either `::` syntax or the `CAST()` function.

Convert `salary` (numeric) to text:

SQL

```sql
SELECT name, salary, salary::TEXT AS salary_text
FROM employees;
```

- `8000` → `'8000'`

Select Statement

PostgreSQL Type Casting

1.2 Using `CAST()`

Same as above:

SQL

```sql
SELECT name, CAST(salary AS TEXT) AS salary_text
FROM employees;
```

2. Common Type Casts

From	To	Example
integer	numeric/text	`age::NUMERIC`
numeric	integer	`salary::INT`
text	integer	`'100'::INT`
text	date	`'2026-03-10'::DATE`
timestamp	date	`NOW()::DATE`

Thank you